

Polymarket 자율 거래 AI 에이전트 구축 가이드북

1. 서론

이 가이드북은 Polymarket 플랫폼에서 자율적으로 거래하는 AI 에이전트를 구축하는 방법에 대한 포괄적인 지침을 제공합니다. 최근 Argona0x라는 X 사용자가 48시간 만에 50달러를 2,980달러로 불린 AI 트레이딩 에이전트의 성공 사례를 공유하며 이러한 기술의 잠재력을 입증했습니다. 이 가이드북은 Argona의 게시물 [1]과 Sapience Builder Guide [2], Polymarket API 문서 [3]를 기반으로 에이전트의 핵심 구성 요소, 구축 단계 및 고급 고려 사항을 다룹니다.

2. Argona의 AI 에이전트 개요

Argona의 AI 에이전트는 Polymarket에서 50달러로 시작하여 48시간 만에 2,980달러를 벌어들였습니다. 이 에이전트의 주요 특징은 다음과 같습니다 [1]:

- 자율성:** 스스로 시장을 스캔하고, 예측을 생성하며, 거래를 실행합니다.
- 주기적 운영:** 10분마다 다음 작업을 수행합니다.
 - 500-1000개 시장 스캔
 - Claude를 사용하여 공정 가치 추정치 구축
 - 8% 이상의 가격 불일치 발견
 - 켈리 기준(Kelly Criterion)을 사용하여 포지션 규모 계산 (최대 банкроলের 6%)
 - 거래 실행
 - 수익으로 자체 API 비용 지불
- 생존 메커니즘:** 잔액이 0달러가 되면 에이전트가 죽도록 설계되어 생존을 학습합니다.
- 기술 스택:** 속도를 위해 Rust로 구축되었으며, 추론을 위해 Claude API를 사용하고, 월 4.5달러의 VPS에서 실행됩니다.
- 데이터 소스:** 날씨 시장의 경우 NOAA 데이터를 파싱하고, 스포츠 시장의 경우 부상 보고서를 스크랩하며, 암호화폐 시장의 경우 온체인 지표와センチ멘트를 분석합니다.

3. Polymarket 트레이딩 에이전트의 핵심 구성 요소

Polymarket에서 작동하는 AI 트레이딩 에이전트를 구축하려면 여러 핵심 구성 요소가 필요합니다.

3.1. 예측 시장 플랫폼 (Polymarket)

Polymarket은 다양한 이벤트에 대한 예측 시장을 제공하는 플랫폼입니다. 사용자는 미래 이벤트의 결과에 베팅하여 수익을 얻을 수 있습니다. AI 에이전트는 이 플랫폼의 API를 통해 시장 데이터에 접근하고 거래를 실행합니다 [3].

3.2. 예측을 위한 AI/LLM (Claude)

Argona의 에이전트는 Claude를 사용하여 시장의 공정 가치를 추정하고 예측을 생성합니다. 이는 LLM(대규모 언어 모델)이 복잡한 정보를 분석하고 확률적 예측을 도출하는 데 사용될 수 있음을 보여줍니다. Sapience 가이드에서는 OpenRouter API를 통해 `openai/gpt-4o-mini`와 같은 LLM을 활용하는 예시를 제공합니다 [2].

3.3. 트레이딩 프레임워크 (Sapience SDK 및 OpenClaw)

Sapience SDK는 예측 시장 트레이딩 에이전트를 구축하기 위한 유용한 도구와 라이브러리를 제공합니다. 이 SDK는 시장 데이터 페치, 예측 생성, 거래 실행 등의 과정을 간소화합니다 [2]. Argona의 게시물에서 언급된 OpenClaw는 AI 에이전트 프레임워크로, 자율적인 워크플로우를 구축하는 데 사용될 수 있습니다 [4].

3.4. 인프라

에이전트는 안정적이고 빠른 실행을 위해 VPS(가상 사설 서버)와 같은 인프라가 필요합니다. Argona의 에이전트는 월 4.5달러의 VPS에서 실행되며, 속도를 위해 Rust로 구축되었습니다 [1].

4. AI 트레이딩 에이전트 구축 단계

Sapience Builder Guide [2]를 기반으로 Polymarket 트레이딩 에이전트를 구축하는 단계는 다음과 같습니다.

4.1. 전제 조건

- **Node.js:** 버전 20.14 이상이 필요합니다.

- **OpenRouter API 키:** LLM 추론을 위해 필요합니다. 다른 LLM 제공업체를 사용할 수도 있습니다.
- **Ethereum 개인 키:** USDe 토큰이 있는 이더리움 개인 키가 필요합니다. Sapience 가이드에서는 Ethereum (체인 ID 5064014)을 예시로 듭니다.

4.2. 환경 설정

1. **프로젝트 디렉토리 생성 및 초기화:** 다음 명령어를 사용하여 프로젝트 디렉토리를 생성하고 pnpm 을 사용하여 초기화합니다.

```
mkdir my-trading-agent
cd my-trading-agent
pnpm init
```

2. **필요한 패키지 설치:** Sapience SDK 및 기타 종속성을 설치합니다.

```
pnpm add @sapience/sdk graphql-request dotenv ws viem
pnpm add -D tsx typescript @types/ws
```

3. **환경 변수 설정:** 프로젝트 디렉토리에 .env 파일을 생성하고 다음 내용을 추가합니다.

```
OPENROUTER_API_KEY=your_openrouter_key
PRIVATE_KEY=your_private_key
```

4.3. 에이전트 로직 구현 (Sapience SDK 기반)

index.ts 파일을 생성하고 다음 로직을 구현합니다.

1. **활성 조건(질문) 가져오기:** Sapience API에서 현재 활성화된 예측 시장 질문을 가져옵니다.

```

import 'dotenv/config';
import { gql } from 'graphql-request';
import {
  encodeAbiParameters,
  encodeFunctionData,
  parseEther,
  type Hex,
  type Address,
} from 'viem';
import { privateKeyToAccount } from 'viem/accounts';
import {
  graphqlRequest,
  CHAIN_ID_ETHEREAL,
  createAuctionWs,
  submitTransaction,
  contracts,
  prepareForTrade,
} from '@sapience/sdk';

const RPC_URL = 'https://rpc.ethereal.trade';
const WS_URL = 'wss://relay.sapience.xyz/auction';
const RESOLVER =
  contracts.pythConditionResolver[CHAIN_ID_ETHEREAL].address;
const PREDICTION_MARKET =
  contracts.predictionMarketEscrow[CHAIN_ID_ETHEREAL].address;
const POSITION_SIZE = '1000000000000000000'; // 1 WUSDe (18 decimals)

const account = privateKeyToAccount(process.env.PRIVATE_KEY as Hex);

async function fetchCondition() {
  const nowSec = Math.floor(Date.now() / 1000);
  const query = gql`
    query Conditions($chainId: Int, $nowSec: Int) {
      conditions(
        where: {
          chainId: { equals: $chainId }
          public: { equals: true }
          endTime: { gt: $nowSec }
        }
        orderBy: { endTime: asc }
        take: 1
      ) {
        id
        question
        shortName
      }
    }
  `;
  const response = await graphqlRequest(RPC_URL, query, {});
  const condition = response.data.conditions[0];
  return condition;
}

export { fetchCondition };

```

```
    }  
  }  
`;  
const { conditions } = await graphqlRequest<{ conditions: any[] }>  
(query, {  
  chainId: CHAIN_ID_ETHEREAL,  
  nowSec,  
});  
return conditions[0];  
}
```

2. **LLM을 사용하여 예측 생성**: LLM(예: `openai/gpt-4o-mini`)을 호출하여 질문에 대한 확률을 예측합니다.

```

async function generateForecast(
  question: string
): Promise<{ probability: number; reasoning: string }> {
  const response = await fetch(
    'https://openrouter.ai/api/v1/chat/completions',
    {
      method: 'POST',
      headers: {
        Authorization: `Bearer ${process.env.OPENROUTER_API_KEY}`,
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({
        model: 'openai/gpt-4o-mini',
        messages: [
          {
            role: 'user',
            content: `Analyze this prediction market question and
provide a probability (0-100) that the answer is YES: "${question}". \
Respond in JSON format: { "probability": <number>,
"reasoning": "<brief explanation>" }`,
          },
        ],
      }),
    },
  );
  const data = await response.json();
  const result = JSON.parse(data.choices[0].message.content);
  return {
    probability: Math.max(0, Math.min(100, result.probability)),
    reasoning: result.reasoning,
  };
}

```

3. **예측 인코딩:** 예측 결과를 거래에 사용될 수 있는 형식으로 인코딩합니다.

```
function encodePrediction(marketId: string, probability: number) {
  const id = (marketId.startsWith('0x') ? marketId : `0x${marketId}`)
  as Hex;
  const prediction = probability > 50; // YES if >50%, NO if <50%

  return encodeAbiParameters(
    [
      {
        type: 'tuple[]',
        components: [
          { name: 'marketId', type: 'bytes32' },
          { name: 'prediction', type: 'bool' },
        ],
      },
    ],
    [[{ marketId: id, prediction }]]
  );
}
```

4. **담보 준비:** USDe를 WUSDe로 래핑하고 거래를 위해 승인합니다.

```
async function prepareCollateral() {
  const { wrapTxHash, approvalTxHash } = await prepareForTrade({
    privateKey: process.env.PRIVATE_KEY as Hex,
    collateralAmount: parseEther('10'), // Prepare 10 WUSDe
  });
  if (wrapTxHash) console.log(`Wrap tx: ${wrapTxHash}`);
  if (approvalTxHash) console.log(`Approval tx: ${approvalTxHash}`);
}
```

5. **거래 실행:** PredictionMarket 컨트랙트의 mint() 함수를 호출하여 거래를 실행합니다.

```

async function executeTrade(
  bid: any,
  encodedOutcomes: Hex,
  auctionTakerNonce: number
) {
  const mintRequest = {
    encodedPredictedOutcomes: encodedOutcomes,
    resolver: RESOLVER,
    makerCollateral: BigInt(POSITION_SIZE),
    takerCollateral: BigInt(bid.makerWager),
    maker: account.address,
    taker: bid.maker as Address,
    makerNonce: BigInt(auctionTakerNonce),
    takerSignature: bid.makerSignature as Hex,
    takerDeadline: BigInt(bid.makerDeadline),
    refCode:

'0x0000000000000000000000000000000000000000000000000000000000000000'
as Hex,
  };

  const { hash } = await submitTransaction({
    rpc: RPC_URL,
    privateKey: process.env.PRIVATE_KEY as Hex,
    tx: {
      to: PREDICTION_MARKET,
      data: encodeFunctionData({
        abi: [
          {
            name: 'mint',
            type: 'function',
            inputs: [
              {
                name: 'mintPredictionRequestData',
                type: 'tuple',
                components: [
                  { name: 'encodedPredictedOutcomes', type: 'bytes' },
                  { name: 'resolver', type: 'address' },
                  { name: 'makerCollateral', type: 'uint256' },
                  { name: 'takerCollateral', type: 'uint256' },
                  { name: 'maker', type: 'address' },
                  { name: 'taker', type: 'address' },
                  { name: 'makerNonce', type: 'uint256' },
                  { name: 'takerSignature', type: 'bytes' },
                  { name: 'takerDeadline', type: 'uint256' },
                ]
              }
            ]
          }
        ]
      })
    }
  })
}

```

```

        { name: 'refCode', type: 'bytes32' },
      ],
    },
    outputs: [],
  },
],
functionName: 'mint',
args: [mintRequest],
}),
},
});

return hash;
}

```

6. **메인 함수:** 위 함수들을 조합하여 에이전트의 주요 로직을 실행합니다.

```

async function main() {
  // 1. Fetch one condition
  console.log('Fetching condition...');
  const condition = await fetchCondition();
  const question = condition.shortName || condition.question;
  console.log(`\nCondition: ${question}`);

  // 2. Generate forecast
  const forecast = await generateForecast(question);
  const position = forecast.probability > 50 ? 'YES' : 'NO';
  console.log(`Forecast: ${forecast.probability}% -
${forecast.reasoning}`);
  console.log(`Trading: ${position}\n`);

  // 3. Prepare collateral (wrap USDe to WUSDe and approve)
  console.log('Preparing collateral...');
  await prepareCollateral();

  // 4. Start auction
  const encodedOutcomes = encodePrediction(condition.id,
forecast.probability);
  const takerNonce = Date.now();
  let auctionId: string | null = null;

  console.log('Starting auction...');
  const ws = createAuctionWs(Ws_URL, {
    onOpen: async () => {
      // Prepare auction payload
      const payload = {
        taker: account.address,
        wager: POSITION_SIZE,
        resolver: RESOLVER,
        predictedOutcomes: [encodedOutcomes],
        takerNonce,
        chainId: CHAIN_ID_ETHEREAL,
      };

      // Optional: Sign the auction payload if required by the relayer
      // const signedPayload = await signAuctionPayload(payload,
account);

      // Send auction request
      ws.send(JSON.stringify({ type: 'AUCTION_REQUEST', payload }));
    },
    onMessage: async (event) => {

```

```

    const message = JSON.parse(event.data as string);
    if (message.type === 'AUCTION_RESPONSE' &&
message.payload.auctionId) {
        auctionId = message.payload.auctionId;
        console.log(`Auction started with ID: ${auctionId}`);
    } else if (message.type === 'BID_RECEIVED' &&
message.payload.bid) {
        const bid = message.payload.bid;
        console.log(`Bid received: ${JSON.stringify(bid)}`);

        // Execute trade with the best bid
        console.log('Executing trade...');
        const txHash = await executeTrade(bid, encodedOutcomes,
takerNonce);
        console.log(`Trade executed with transaction hash:
${txHash}`);

        ws.close(); // Close WebSocket after trade
    }
},
onClose: () => console.log('WebSocket closed'),
onError: (error) => console.error('WebSocket error:', error),
});
}

main().catch(console.error);

```

4.4. Polymarket API 통합 및 전략

Polymarket API는 시장 데이터를 스캔하고 거래를 관리하는 데 사용됩니다 [3].

- **시장 스캔:** `getOpenOrders` 와 같은 API를 사용하여 현재 열려 있는 시장과 오더북을 스캔할 수 있습니다. Argona의 에이전트는 500-1000개의 시장을 스캔하여 가격 불일치를 찾습니다.
- **주문 생성 및 관리:** `createAndPostOrder` 를 사용하여 단일 주문을 생성하거나, `postOrders` 를 사용하여 여러 주문을 일괄 처리할 수 있습니다. `cancelOrder` , `cancelOrders` , `cancelMarketOrders` , `cancelAll` 을 사용하여 주문을 취소할 수 있습니다.
- **틱 사이즈 및 수수료:** 시장의 틱 사이즈를 확인하여 올바른 가격으로 주문을 제출해야 합니다. 시장 조성자(maker)는 수수료가 부과되지 않지만, 시장 참여자(taker)는 수수료가 부과될 수 있습니다.

- **최적의 전략:** Polymarket 문서에서는 다음과 같은 모범 사례를 권장합니다 [3].
 - **양방향 호가:** 유동성 보상을 극대화하기 위해 매수(bid)와 매도(ask)를 모두 게시합니다.
 - **재고에 따른 왜곡:** 현재 포지션에 따라 호가 가격을 조정하여 노출을 관리합니다.
 - **오래된 호가 취소:** 시장 상황이 변경되면 즉시 주문을 철회합니다.
 - **GTD(Good-Til-Date) 사용:** 시장 마감 또는 정산과 같은 알려진 이벤트 전에 호가를 자동으로 만료시켜 오래된 노출을 방지합니다.
 - **배치 주문:** `postOrders()` 를 사용하여 여러 호가를 단일 요청으로 제출하여 지연 시간을 줄입니다.
 - **WebSocket 사용:** REST 엔드포인트를 폴링하는 대신 실시간 피드를 구독하여 데이터를 얻습니다.
 - **위험 관리:** 토큰 잔액을 확인하고, 가격 가드를 설정하며, 오류 발생 시 `cancelAll()` 을 호출하는 킬 스위치를 구현합니다.

5. 고급 고려 사항

5.1. Kelly 기준 (Kelly Criterion)을 이용한 포지션 규모 결정

Argona의 에이전트는 Kelly 기준을 사용하여 포지션 규모를 결정합니다 [1]. Kelly 기준은 장기적인 자산 성장을 극대화하기 위한 최적의 베팅 규모를 계산하는 공식입니다. 이는 에이전트의 банкроल(자본)을 효과적으로 관리하고 위험을 최소화하는 데 중요합니다.

5.2. 다양한 데이터 소스 통합

Argona의 에이전트는 날씨 시장을 위해 NOAA 데이터, 스포츠 시장을 위해 부상 보고서, 암호화폐 시장을 위해 온체인 지표 및センチメント 등 다양한 외부 데이터 소스를 파싱합니다 [1]. 이러한 다양한 데이터를 통합하여 LLM의 예측 정확도를 높일 수 있습니다.

5.3. 생존 메커니즘 및 학습

잔액이 0달러가 되면 에이전트가 죽도록 설계된 Argona의 접근 방식은 에이전트가 ‘생존’을 학습하도록 유도합니다 [1]. 이는 강화 학습(Reinforcement Learning)과 유사하게 에이전트가 손실을 피하고 수익을 창출하는 전략을 스스로 발전시키도록 장려할 수 있습니다.

6. 결론

Polymarket에서 자율 거래 AI 에이전트를 구축하는 것은 복잡하지만 매우 보람 있는 작업입니다. Sapience SDK와 Polymarket API를 활용하고, LLM을 통한 정교한 예측 모델을 구축하며, 효과적인 위험 관리 전략을 통합함으로써 Argona의 사례와 같이 성공적인 에이전트를 개발할 수 있습니다. 이 가이드북이 여러분의 AI 트레이딩 에이전트 구축 여정에 도움이 되기를 바랍니다.

7. 참고 자료

1. [Argona on X: “i gave an AI 50andtoldit”payforyoursel foryoudie”48hourslaterittturned50 into \\$2,980 and it’s still alive autonomous trading agent on polymarket every 10 minutes it: → scans 500-1000 markets → builds fair value estimate with claude → finds mispricing > 8% https://t.co/E1H4L8mZVh” / X](#)
2. [Deploy a Trading Agent – Sapience](#)
3. [Polymarket Documentation](#)
4. [GitHub - openclaw/openclaw: Your own personal AI assistant. Any OS. Any Platform. The lobster way. 🦞 · GitHub](#)